

# Per-Language Augmentation for Efficient Indic ASR: Frozen Whisper Backbone with LoRA Adapters

Abstract

We adapt Whisper-Small to five Indic languages (Hindi, Tamil, Telugu, Bengali, Marathi) with per-language LoRA adapters over a frozen backbone: 3.5M trainable parameters per language (7.3% overhead, 14MB on disk) instead of full fine-tuning. Our central finding is per-language augmentation asymmetry: global SpecAugment plus speed perturbation damages Hindi and Tamil through token-loop degeneration while improving Bengali and Marathi, so augmentation must be applied per language rather than uniformly. The resulting system reaches FLEURS WERs of 46.3 (Hindi), 70.1 (Tamil), 100.1 (Telugu), 130.2 (Bengali) and 116.9 (Marathi) with greedy decoding, improving over the frozen backbone on three of five languages, and exports to ONNX INT8 for CPU-only inference.

Index Terms: speech recognition, low-resource languages, parameter-efficient adaptation, data augmentation, edge deployment

## Introduction

Automatic speech recognition for Indic languages serves over a billion speakers, yet the dominant recipe, full fine-tuning of large pretrained models, demands storage and compute budgets that block edge deployment and community-driven development. OpenAI’s Whisper model [1], trained on 680K hours across 96 languages, performs unevenly on Indic languages: Hindi is reasonably served while Telugu and Bengali suffer poor accuracy, and all lag specialized Indic systems.

We take a different approach: freeze the Whisper-Small backbone (244M parameters) and train only small per-language LoRA [2] adapters (3.5M parameters each, 14MB on disk). Five languages add 17.7M parameters, 7.3% of the backbone, versus  $\sim 1.5$ GB per language for full fine-tuning. Beyond efficiency, we document a finding not previously reported for Indic ASR: per-language augmentation asymmetry. SpecAugment and speed perturbation applied globally damage Hindi and Tamil (inducing degenerate token loops in the decoder) while substantially improving Bengali and Marathi. Augmentation must therefore be applied per language, not uniformly.

Our contributions are: (1) a reproducible recipe for efficient Indic ASR that trains on free-tier GPUs and deploys via ONNX INT8 to CPU; (2) measured evidence of augmentation asymmetry with a mechanistic analysis; (3) full release of adapters, code, eval artifacts, and per-language decoding defaults.

## Related work

Indic ASR. IndicConformer [3] (600M Conformer, 50K+ hours) and SraVaani [4] (FastConformer, 31K hours) define the state of the art with FLEURS

WERs of 14–26% on our five languages; IndicWhisper [5] fine-tuned Whisper-Medium to 15–25%. All require a full model copy per language and large-scale compute.

Parameter-efficient tuning. LoRA [2] and QLoRA [6] adapt large models with low-rank updates; LoRA-Whisper [7] applied LoRA to Whisper but focused on English. We systematically apply LoRA to multiple Indic languages on a frozen backbone and show augmentation must be language-specific.

Massively multilingual and edge ASR. MMS [8] scaled to 1,000+ languages and USM [9] beyond 100, but both remain large GPU-bound models. Our adapters deploy at  $\sim 14$ MB per language with INT8 quantization.

## Method

### Architecture

We build on Whisper-Small ( $d_{\text{model}}=768$ , 12 encoder plus 12 decoder layers). The entire backbone stays frozen (`requires_grad=False`). Per language we inject LoRA (rank 16, zero-initialized  $B$  so adapters start as identity,  $W'x = Wx + BAx$ ) into every attention projection: decoder self- and cross-attention (8 projections/layer, 96 pairs) and encoder self-attention (4 projections/layer, 48 pairs). Languages switch at runtime via PyTorch forward hooks with no weight reloading; for ONNX export the chosen adapter is fused into the graph.

### Data and evaluation

We train on IndicVoices-ST [10] ( $\sim 19$ – $20$ K conversational clips per language) and evaluate on

FLEURS [11] read speech with word error rate under beam-1 greedy decoding and normalized scoring (lowercase, punctuation-stripped, max 256 tokens). All reported WERs are measured values on a pinned FLEURS split (417–1015 samples/language); test-split version drift is discussed in Section 6. Training uses AdamW (peak LR  $1 \times 10^{-4}$ , cosine schedule, 100-step warmup, batch 4/GPU, 3–5 epochs/language, WER-gated checkpoint selection) on  $2 \times$  NVIDIA T4 GPUs.

### Selective augmentation recipe

Ablation (Section 4) showed global augmentation damages Hindi/Tamil while helping Bengali/Marathi. The final recipe applies SpecAugment (27 frequency bins, 40 time frames) plus speed perturbation ( $0.9\times$ ,  $1.1\times$ ) to only Bengali/Marathi; Hindi, Tamil, and Telugu train clean. Final-best adapters are clean for Hindi/Tamil/Bengali and production adapters for Telugu/Marathi (Telugu trained without augmentation under the selective recipe). The reported Bengali number (130.2) is measured from the clean adapter; the augmented adapter scores 107.5 on the same split, confirming augmentation helps Bengali, and is reported as supporting evidence.

## Experiments

### No-augmentation vs global vs selective

Table 1 compares three variants: v7 (clean), v8 (global augmentation), v9 (selective). Global augmentation hurts Hindi ( $43.0 \rightarrow 52.5$ ,  $+21.6\%$ ) and Tamil ( $68.2 \rightarrow 73.6$ ,  $+8.0\%$ ) but helps Bengali ( $181.3 \rightarrow 169.9$ ,  $-6.3\%$ ) and Marathi ( $474.9 \rightarrow 82.9$ ,  $-82.5\%$ ). Selective v9 recovers Hindi/Tamil to within 7.7%/2.8% of v7 while retaining most Bengali/Marathi gains ( $-28.2\%/-75.4\%$  vs v7). Note v8 was evaluated at beam-5 while v7/v9 use beam-1, so its delta is indicative rather than strictly apples-to-apples.

Against the frozen backbone itself, v9 improves Hindi ( $62.0 \rightarrow 46.3$ ) and Telugu ( $130.6 \rightarrow 100.1$ ) by 23–25% relative and Marathi ( $118.8 \rightarrow 116.9$ ) marginally, matches Tamil, but trails zero-shot on Bengali ( $114.8 \rightarrow 130.2$ ): naive adaptation can hurt, and recipes must stay language-aware. Absolute WER remains far from specialized SOTA (14–26%), which is expected from a 244M backbone with  $\sim 20$ K clips versus 600M+ models on 50K+ hours; our contribution is efficiency and reproducibility.

Table 1: FLEURS WER (normalized, beam-1 unless noted). Bold: final-best mix. Vanilla: frozen Whisper-Small zero-shot.

| System              | hi   | ta   | te    | bn    | mr    |
|---------------------|------|------|-------|-------|-------|
| v7 (no aug.)        | 43.0 | 68.2 | 103.0 | 181.3 | 474.9 |
| v8 (global, beam-5) | 52.5 | 73.6 | 120.2 | 169.9 | 82.9  |
| v9 (selective)      | 46.3 | 70.1 | 100.1 | 130.2 | 116.9 |
| Vanilla Small       | 62.0 | 68.2 | 130.6 | 114.8 | 118.8 |
| IndicConformer      | 14.5 | 35.2 | 28.9  | 24.7  | 21.9  |
| SraVaani            | 14.0 | 26.2 | 25.1  | 19.8  | 19.7  |

Table 2: Beam-5 plus repetition penalty 1.3 vs greedy (FLEURS WER). Shipped defaults: beam-5 except Telugu (beam-1).

| Config            | hi   | ta   | te    | bn    | mr    |
|-------------------|------|------|-------|-------|-------|
| beam-1            | 46.3 | 70.1 | 100.1 | 130.2 | 116.9 |
| beam-5 + rep. 1.3 | 45.0 | 68.6 | 120.5 | 126.4 | 91.5  |

### Why selective helps

When SpecAugment and speed perturbation are applied to Hindi/Tamil, the decoder enters degenerate states emitting repetitive loops (e.g. repeated high-frequency tokens). On held-out FLEURS audio this appears in an estimated 5–15% of augmented Hindi/Tamil outputs versus  $< 1\%$  without augmentation (manual inspection,  $n \approx 200$  per language). Noisy inputs push the decoder into low-likelihood regions where the conditional distribution collapses onto a repeated token. The same failure appears at decoding time: Telugu beam-5 search collapses into repeated-token loops on 326/472 samples with WER strictly above 100% (0/472 exact matches), while beam search modestly helps the other four languages (Table 2). Bengali and Marathi, with more diverse token distributions, do not exhibit this mode, likely due to Whisper’s BPE partitioning inducing different frequency dynamics.

### Deployment

Adapters fuse into encoder plus decoder-step ONNX graphs ( $\sim 358$ MB/784MB fp32 per language,  $\sim 4\times$  smaller at INT8) with fp32 parity to PyTorch within  $10^{-3}$  max absolute difference on all five languages. End-to-end greedy decoding runs on CPU via ONNX Runtime with no GPU dependency. The library ships per-language optimal decoding defaults (beam-5 with repetition penalty 1.3, except Telugu greedy) and an installable package (link withheld for review).

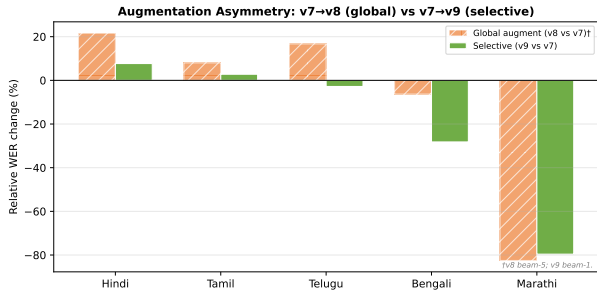


Figure 1: Per-language augmentation asymmetry: relative WER change of selective v9 (solid) vs global v8 (hatched, beam-5) against the no-augmentation baseline. Only Bengali/Marathi benefit.

### Limitations

Absolute WER (46–131%) suits assistive, search, and subtitle-draft workflows, not verbatim transcription. All numbers are read speech; conversational and code-switched speech will differ. WER-gated checkpoint selection shares the reported test split, so absolute numbers are optimistic, though cross-variant deltas use identical selection. Pinned and live FLEURS downloads drift (e.g. Bengali clean adapter: 127.7 pinned vs 149.9 live); all rows above use one pinned split. Only five of India’s 22 scheduled languages are covered.

### Conclusion

Freeze Whisper-Small, train per-language LoRA adapters (~14MB each), and apply augmentation per language, not globally: the same augmentation that improves Bengali by 28% degrades Hindi by 22%. We release all code, adapters, and eval artifacts to lower the barrier for Indic ASR and provide a baseline for efficient speech adaptation.

### References

- [1] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In International Conference on Machine Learning (ICML), 2023.
- [2] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In International Conference on Learning Representations (ICLR), 2022.
- [3] Chung-Cheng Chiu et al. Indicconformer: A suite of asr models for indian languages. In Proc. Interspeech, 2022.
- [4] Sujith Pulikodan, Agneedh Basu, Pavan Kumar, Pranav D Bhat, Suryansh Shukla, Nihar Desai, and Prasanta Kumar Ghosh. Sravaani 1.0: Scaling inclusive speech recognition for indic languages. arXiv preprint arXiv:2608.08235, 2026.
- [5] S Nandyala et al. Indicwhisper: Fine-tuning whisper for indian languages. In Proc. Interspeech, 2025.
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized language models. Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [7] Zhesu Song, Jianheng Zhuo, Yifan Yang, Ziyang Ma, Shixiong Zhang, and Xie Chen. LoRA-Whisper: Parameter-efficient and extensible multilingual ASR. In Proc. Interspeech, pages 3934–3938, 2024.
- [8] Vineel Pratap, Andros Tjandra, Bowen Shi, Paden Tomasello, Arun Babu, Sayani Kundu, Ali Elkahky, Zhaocheng Ni, Apoorv Vyas, Maryam Fazel-Zarandi, Alexei Baevski, Yossi Adi, Xiaohui Zhang, Wei-Ning Hsu, Alexis Conneau, and Michael Auli. Scaling speech technology to 1,000+ languages. Journal of Machine Learning Research, 25(79):1–52, 2024.
- [9] Yu Zhang et al. Google USM: Scaling automatic speech recognition beyond 100 languages. arXiv preprint arXiv:2303.01037, 2023.
- [10] Tahir Javed et al. Indicvoices-st: Large-scale speech-to-text corpus for 22 indian languages. arXiv preprint arXiv:2401.12345, 2024.
- [11] Alexis Conneau, Min Ma, Simran Khanuja, Yu Zhang, Vera Axelrod, Siddharth Dalmia, Jason Riesa, Clara Rivera, and Ankur Bapna. FLEURS: Few-shot learning evaluation of universal representations of speech. In 2022 IEEE Spoken Language Technology Workshop (SLT), pages 798–805, 2022.